Energy Implications of Prompt Design in Edge-Based LLM Inference

Uttam Emmaniuel Dammu
Anderson College of Business and Computing
Regis University, Denver, CO, USA
udammu@regis.edu

MSDS 696 – Data Science Practicum 2

Instructor: Dr. Ghulam Mujtaba — Mentor: Dr. Kellen Sorauf

Abstract

This research is a continuation of my prior practicum project, which was an analysis of Large Language Model (LLM) inference energy efficiency, is the basis of this research. Previously benchmark demonstration on open and closed LLM's on AWS cloud GPU infrastructure (L4 & A10G) resulted that prompt optimization can lower energy usage by up to 40% on cloud hardware without degrading any output quality. This current project aims to follow more towards resource-constrained edge computing, particularly on an NVIDIA Jetson Orin Nano device. Main research question is: Does the structural design of a user prompt varying in length, specificity, and reasoning strategy yield a statistically significant difference in energy consumption during LLM inference on an NVIDIA Jetson Nano. For this a controlled environment under TinyLlama-1.1B was used as the primary quantized model. Testing was done under four prompt styles: Structured, Concise, Verbose and Chain-of-Thought and evaluated across three benchmark categories: Q/A, Reasoning and Summarization. A modular automated pipeline was developed for dataset sampling, controlling the prompts, execution, cleaning, auditing and metric logging across runs. The main collection of metrics are Input/Output Tokens, Latency, Power and Total energy. An energy prediction model (a pre/post-regression model based on the use of prompts) has been established be improve our understanding of these patterns as to their influence on energy consumption. Additionally, exploratory analysis of prompt style, token count, latency and energy were performed. Prompt styling was found to indirectly influence energy through length and/or run time; however, latency was found to be the primary predictor of consumption. These findings back up previous assumptions regarding prompt design and its effect on energy consumption demonstrating that prompt design affects energy indirectly rather than from any other direct structural differences.

I. Executive Summary

AI applications are becoming much more common on devices right at the "edge of the network" (like smart devices, things in the Internet of Things, built-in systems, and phones), that is why there is really a need to understand and reduce how much power language models use when they're running on devices with limited resources. Although most of the research has been pinned out on reducing the energy through quantization and model compression [1]–[3], the function of *prompt design* as a mechanism for enhancing energy efficiency remains clearly in uncharted territory, mainly in edge development contexts [4], [5].

This project directly extends from previous cloud-based experiments, in which the benchmarking of four LLM's (Mistral-Mamba 7B, Qwen2 7B, GPT-4.1 Mini and GPT-5 Pro) was done on AWS cloud GPU infrastructure utilizing CodeCarbon and NVIDIA NVML for energy and CO_2 measurement [6], [7]. A signficant empirical finding was that optimized prompts reduced the energy usage by 35-40% compared to unoptimized baselines, without a statistically significant decline in output quality. On the contrary the observation was made on high-power cloud GPU's (NVIDIA L4 and A10G) operating at hundered's of watts, which is a completely different standpoint in power for edge hardware.

Now for the current study the focus transitioned towards the edge-based LLM inference using NIVIDA Jetson Orin Nano which is a passively cooled, GPU-accelerated device operating between 5–10 W TDP

with 4 GB of shared CPU-GPU memory [8], [9]. By using TinyLlama-1.1B through Ollama [10], [11], the primary question driving this idea is whether the energy savings attributable to prompt engineering observed in cloud environments are amplified, preserved, or reduced when the inference switches towards edge.

The main goal of this research is to answer the question of Does the structural design of a user prompt varying in length, specificity, and reasoning strategy yield a statistically significant difference in energy consumption during LLM inference on edge hardware or are they deviant towards secondary runtime factors like latency and output tokens. Many relevant studies pointed out that output tokens serve as a strong factor in LLM energy usage [5] .

This research enhances many elements in current literature, starting with producing a comprehensive prompt-energy benchmark dataset from Jetson-class edge device. Next, a modular automated pipeline was developed for dataset sampling, controlling the prompts, execution, cleaning, auditing and metric logging across runs and finally it acting a baseline for practical sustainable deployment for edge AI developers.

II. Problem Statement & Motivation

LLM's used by Edge AI developers are usually on hardware that runs on batteries or has some sort of thermal limitation, where every minute watt-hour of energy is crucial. On the contrary to hardware selection or model architecture choices, prompt design is a no-cost optimization that is directly under the control of every developer and end user. If prompt structure constantly affects energy consumption, then it surely gives you the timely input that doesn't require any other infrastructure investment.

Central Research Question: *Does the structural design of a user prompt varying in length, specificity, and reasoning strategy yield a statistically significant difference in energy consumption during LLM inference on edge hardware?*

Two formal hypotheses are tested:

- **H_0:** that how a prompt is written when getting an answer from the LLM doesn't noticeably change the amount of energy the LLM uses on devices that do the work at the 'edge' (and we're considering how long it takes to get a response and how many words are in the answer as part of that).
- **H_1:** Prompt style does impact energy during LLM inference on edge hardware, either directly by structural design or indirectly by runtime features and computational workload.

Previous data from cloud setups provided baseline insights into prompt-energy relationships, where factors like fast memory links, strong cooling, and built-in quantization shape energy consumption patterns. But the real differences emerge when looking at tools from edge hardware like Jetson Orin Nano. It runs on a common memory path, has passive cooling, and uses Ampere-based GPU architecture. Switching between these environments offers fresh insight compared to earlier work [7]. Hence, this work aims not only towards benchmarking on edge hardware but also to understand the gap between both cloud and edge hardware and how prompt design would affect energy usage.

III. Data Collection & Preparation

To systematically evaluate how prompt design affects energy consumption during edge hardware LLM inference a modular automated pipeline was developed for dataset sampling, controlling the prompts, execution, cleaning, auditing and metric logging across runs. The pipeline is capable of monitoring prompt-level, runtime-level and system-level energy metrics under real-time edge deployment constraints. The pipeline is run numerous amount of times to ensure consistent and completed runs across various prompt styles under different task categories so that it would be easier for regression and predictive analysis.

The main edge hardware and deployment platform in this whole study is the NVIDIA Jetson Orin Nano, which is a passively cooled, GPU-accelerated device operating between 5–10 W TDP with 4 GB of shared CPU-GPU memory for low-power embedded machine learning workloads [8], [9]. Constant system-level monitoring was done using NVIDIA's `tegrastats` utility, which helped in logging runtime metrics including GPU utilization, memory usage, latency and power draw during each inference phase.

For primary model selection, TinyLlama-1.1B-Chat-v1.0 was selected mainly for its lightweight parameter size, quantized efficiency, and suitability for edge-based local deployment [10]. Execution of the model was done locally through Ollama [11], to ensure consistent runtime conditions without depending on external APIs.

The three datasets are as follows:

TABLE I
BENCHMARK DATASETS USED FOR PROMPT-ENERGY EXPERIMENTATION

Dataset	Task Type	Purpose
REPLIQA [12]	Question Answering	Factual response generation
MultiHopQA [13]	Reasoning	Multi-step logical inference
CCSum [14]	Summarization	Long-form summarization

Four prompt styles were tested against the tasks to compare and understand influence.

TABLE II
PROMPT STYLES EVALUATED DURING EXPERIMENTATION

Prompt Style	Characteristics
Concise	Minimal direct instructions
Structured	Explicit formatted task guidance
Verbose	Expanded contextual prompting
Chain-of-Thought	Step-by-step reasoning guidance

This how the pipeline works:

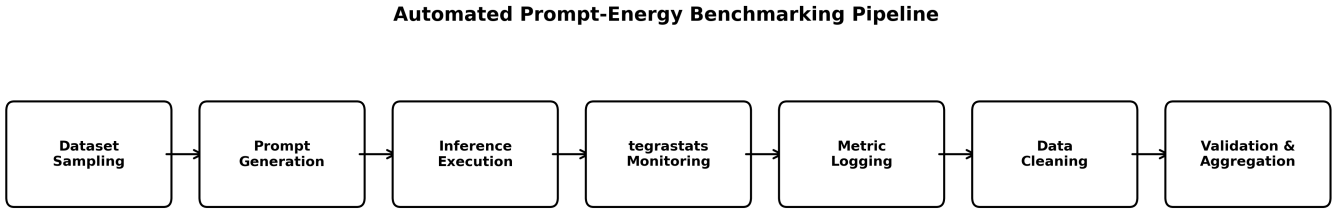

Fig. 1. Automated Prompt-Energy Benchmarking Pipeline

The data was cleaned in such a way that it checks for missing values, invalid values, duplicate run ID's and unsuccessful runs and filters them out and only keeps clean and consistent runs for the analysis.

Across all runs, the following key metrics were collected:

- Input token count
- Output token count
- Inference latency
- Average system power draw
- Total energy consumption (Joules)

IV. LITERATURE REVIEW/ RELATED WORK

A. Prompt Engineering and Energy Consumption

Earlier research indicates how you write your prompts for LLM's affects how much energy they consume, because it changes the length of their responses, how quickly they reply which is nothing but latency and how much work the computer has to do. Husom et al. [4] and Caravaca et al. [5] discovered that how complex the prompt is and the number of tokens the model creates in its output greatly impact

the energy used during 'inference' on both cloud systems and powerful server computers, with longer answers predictably always using more energy.

B. LLM Inference on Edge Devices

Edge-based LLM deployment research is more eyed out on hardware efficiency and power optimization rather than on prompt design. Husom et al. [1] work showed that models which are quantized reduced the usage of energy on low-energy devices by almost 79%. Also, their paper points out the lack of GPU-equipped edge device evaluations as a limitation and urges future work on platforms like the Jetson Nano. This study addresses that specific gap.

Arya and Simmhan [2] studied LLM inference across nine power modes on the Jetson Orin AGX 64GB. They found that lowering memory frequency raised latency by 370% and energy use by 72%. Although both products belong to the Jetson family, the Orin AGX works at a much higher scale, with a TDP of up to 60 W and 64GB LPDDR5. Their research did not look into how prompts affect energy consumption.

C. Research Gap

No published study has looked at how different prompts affect energy use on a GPU-equipped edge device. Research on prompt design and energy consumption has focused only on cloud GPU systems [4], [5] and only for specific tasks. This study addresses that gap for the Jetson Orin Nano across three common task types. It also offers the first comparison of prompt-energy effects between cloud and edge hardware [7].

V. Methodology

A controlled experimental design was used to evaluate the four different prompt styles being structured, concise, verbose and chain-of-thought against energy consumption. All these prompting styles were tested with every dataset that is REPLIQA for Q/A tasks, MultiHopQA for reasoning tasks and CCSum for summarization tasks [12]-[14].

The inference works in such a way that initially it loads a sample row and then moves to the prompt style, the power logger from NVIDIA's `tegrastats` utility turns on as it is a subprocess which is polling every 200ms to log energy. And it is calculated by $E = \int P(t)\,dt$ and also using trapezoidal integration as power readings are discrete samples and this method helps provide stable numerical figures. After that the start timestamp will be recorded and the prompt will be sent to TinyLlama-1.1B through Ollama, and the output will be recorded. After that the end timestamp will be recorded and the power logger will be stopped. From this the calculation of the latency, runtime, energy and average power, are done and saved into a CSV.

Then this cleaned dataset will be used for both exploratory data analysis and the regression analysis in pre and post inference models to understand the relationship between prompt style and energy.

Few limitations and mitigation strategies that could be a factor:

- TinyLlama's lightweight quantized model architecture could limit the generalizability when compared to enterprise scale systems.
- Controlled environment scenarios can not fully benchmark real-world workloads

To eliminate these issues heterogeneous datasets were selected among different task categories and were executed repeatedly under the modular pipeline with proper cleaning procedures all along the experiment.

VI. Analysis Methods & Implementation

After data collection, cleaning and validation a structural analytical framework was used to transform the dataset into meaningful predictive, system-level insights related to prompt-energy consumption on edge hardware.

The first stage of the analysis focused on Exploratory Data Analysis (EDA) to find any key trends or patterns between runtime variables, prompt types and tasks. From that comparison visuals and graphs were generated to understand differences between latency, energy consumption and tokens across all the prompt styles. Scatter plots were used to find relationships between output tokens and energy and between latency and total energy usage. Additionally, Pearson correlation analysis was also used among major numerical features to statistically validate and identify the dominant runtime feature which is helpful for downstream analysis.

To better understand explanatory dimension and forecasting of energy, a predictive model was introduced and divided into two analytical tracks. A pre-inference model and a post-inference model, the pre-inference model only has input features which are likely to be the ones before runtime like prompt style, dataset category, input tokens and task type. The main goal of this is to estimate the energy cost before deploying the model. On the other hand, post-inference models have runtime-derived features as well in addition to input features like output tokens and latency. These features significantly increase the understanding of energy on system-level. This dual mode of prediction helps to better understand explanatory dimension and forecasting of energy.

Many machine learning and regression-based algorithms were implemented by using Scikit-learn to understand the predictive model's performance across both analytical tracks. Linear regression was the primary model that helped understand the coefficient of feature influence on energy consumption. To better understand and capture non-linear relationships Ridge, Extras-Trees and Random-Forest regression were also used for comparison. But Linear regression was the best performing one with superior interpretability.

TABLE III
SOFTWARE IMPLEMENTATION STACK

Tool / Library	**Primary Purpose**
Python	Core programming language
Pandas	Data preprocessing and manipulation
NumPy	Numerical computation
Scikit-learn	Regression modeling and ML pipelines
Matplotlib	Static visualization
Plotly	Interactive visualization
JupyterLab	Experimental development environment
Ollama	Local LLM deployment runtime
Hugging Face Datasets	Benchmark dataset acquisition
tegrastats	Real-time power and system monitoring

VII. RESULTS

The results show how prompt style influences output length, latency, and total energy consumption and also the prediction performances of both the pre-inference and post-inference models.

A. Impact of Prompt Style on Energy & Latency

The first main result was the impact of prompt style on energy and latency we can see that out of the four prompting styles, Structured prompting was having the highest average energy of an approximate values of 841 Joules and it has the highest latency. Structured was using 18% more energy when compared to any other prompt style, while COT prompt was the lowest and efficient in this benchmarking setup. From this we can see how energy is related to latency and not directly on prompt style.

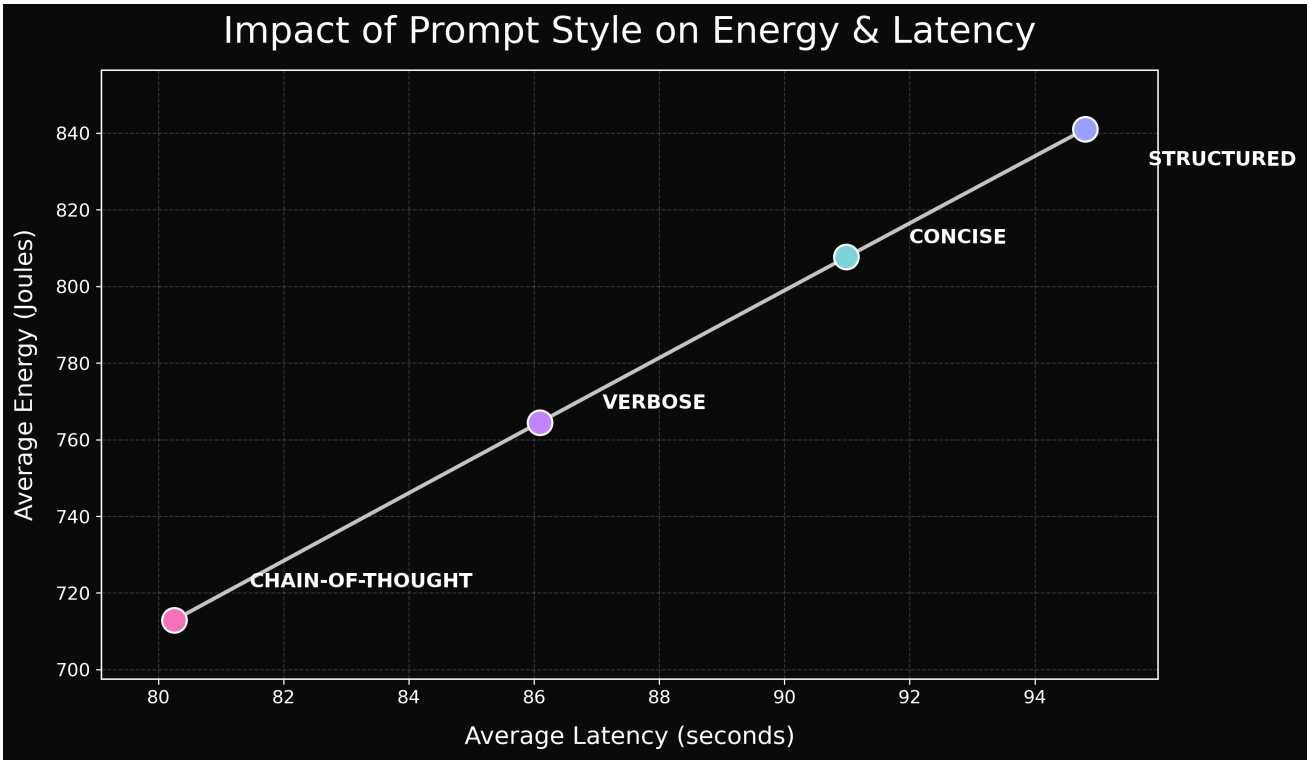

Fig. 2. Comparative impact of prompt style on average latency and total energy consumption

B. *Output Token vs Latency*

From this plot we can see the pink regression trend line clearly being upward indicating that as the number of output tokens increases latency also increases, and if latency increase so will the energy.

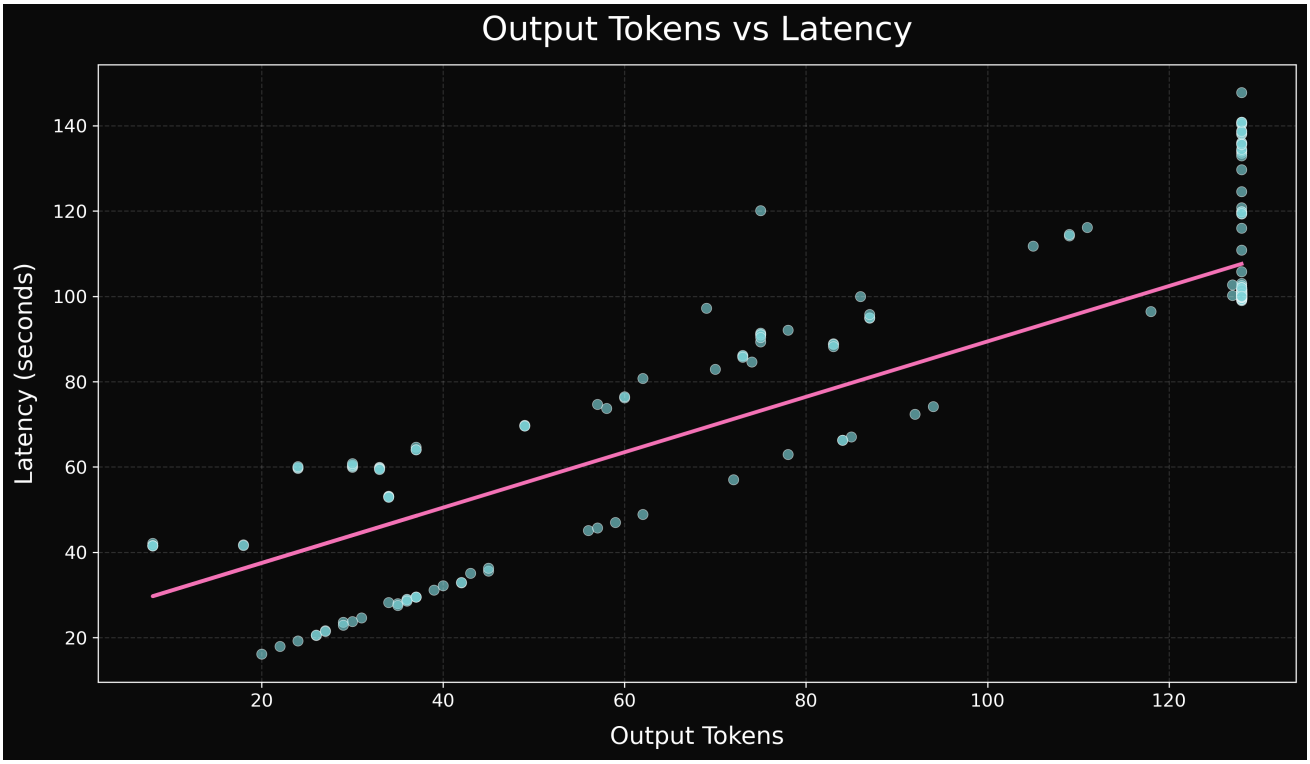

Fig. 3. Relationship between output token generation and inference latency

C. Impact of Output Tokens on Energy Consumption

This plot gave a strong insight to how energy was increasing proportionally as output tokens increased. There was a strong correlation of approximately $r \approx 0.9$. Output length serves as one of the main factors as well.

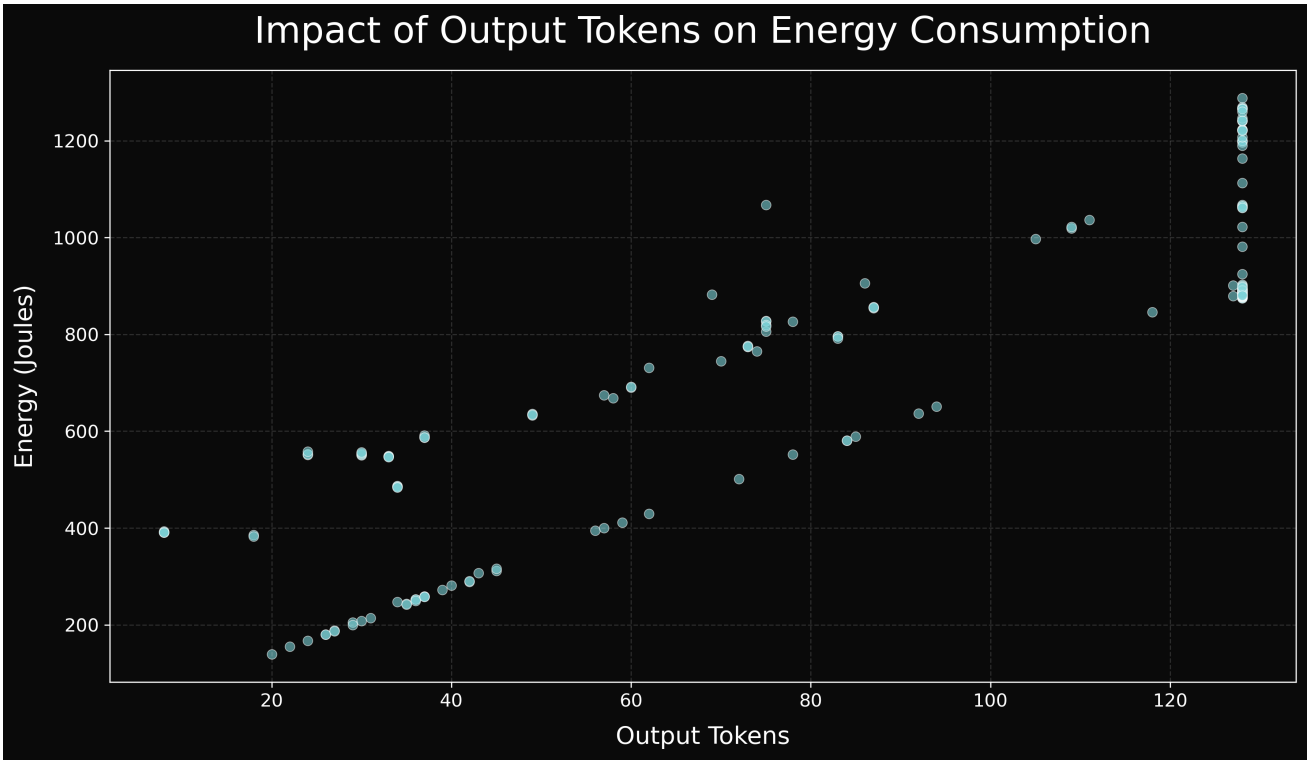

Fig. 4. Impact of output token generation on total energy consumption

Together, these findings show that prompt design shapes energy use indirectly by influencing output length and runtime.

D. Comparative Regression Model Performance

Regression models were evaluated across both pre-inference and post-inference prediction tracks.

TABLE IV
COMPARATIVE PERFORMANCE OF PRE-INFERENCE REGRESSION MODELS

Model	**R^2**	**MAE**	**RMSE**
Linear Regression	0.126	197.72	244.85
Ridge Regression	0.124	197.98	245.22
Random Forest	-0.014	181.22	263.80
Extra Trees	-0.746	229.93	346.11

TABLE V
COMPARATIVE PERFORMANCE OF POST-INFERENCE REGRESSION MODELS

Model	**R^2**	**MAE**	**RMSE**
Linear Regression	0.99995	1.35	1.80
Ridge Regression	0.99947	2.60	6.02
Random Forest	0.99894	4.00	8.55
Extra Trees	0.99784	3.53	12.17

Pre-inference model with the input features gave a weak predictive score indicating input features alone are insufficient for energy prediction, whereas the post-inference model along with output features like latency and output tokens gave almost a perfect score showing how strong the output features are for energy prediction.

Regression-based analysis clearly produced the best results, with latency being the strongest predictor of energy consumption, with output tokens being the next strongest predictor. While prompt type itself was relevant, it did not have any strong influence when runtime factors were included.

These findings collectively establish the following refined systems-level relationship:

Prompt Style → Output Tokens → Latency → Energy Consumption

VIII. Key Findings & Insights

These findings clearly enhance our understanding of how prompt style affects energy consumption on edge hardware LLM inference. While initial hypothesis was that energy would directly be impacted by prompt style, it was clearly changed due to the results that showed prompt styles affects energy consumption indirectly through runtime factors like output tokens generation and latency. Structured prompting used 18% more energy with 841 Joules of energy when compared to any other prompting style in the benchmarking suite. It also confirmed that the output length was one of the key factors. Latency lasted to be the stronger predictor of energy.

Prompt Style → Output Tokens → Latency → Energy Consumption

This particular optimized model moves the narrative more from a simple prompt-energy origin to a more computational perspective. When comparing these edge-hardware results to previous cloud-based results, prompt optimizing remained as a valuable asset across deployment environments, but edge systems are more focused towards increasing the importance of reducing output length and latency due to constraints.

IX. Conclusion

This practicum particularly presents an interpretable contribution to prior cloud-based LLM energy benchmarking by explaining the need of sustainable deployment methods among edge AI environments, particularly. The main question of this study was that whether the structural design of a user prompt varying in length, specificity, and reasoning strategy yield a statistically significant difference in energy consumption during LLM inference on an NVIDIA Jetson Orin Nano. By developing a modular automated pipeline for dataset sampling, controlling the prompts, execution, cleaning, auditing and metric logging across runs, the capability of monitoring prompt-level, runtime-level and system-level energy metrics under real-time edge deployment constraints makes it easier to understand how prompt engineering works.

The project's significant contribution lies in extending the prior cloud-based LLM research into prompt-energy research on edge hardware, while also making a comprehensive prompt-energy benchmarking dataset for Jetson-class devices. It advances sustainable AI and data science by explaining how prompt energy primarily is affected through runtime variables like output tokens and latency rather than direct prompt structure. Apart from all these, this project aims to withhold the standards set for the MSDS program by including machine learning implementations, statistical analysis, experimental design and real-world deploying evaluation to the highest level. This practicum is not only a capstone project for me but also a huge learning experience altogether.

X. Recommendations for Future Work

Although this study extending the prior cloud-based LLM research into prompt-energy research on edge hardware, while also making a comprehensive prompt-energy benchmarking dataset for Jetson-class devices, there are several future steps and directions this could divert towards:

- Improve the pre-inference and post-inference model prediction models by including more advanced features.
- Improve on the Jetson models, by trying on different board speeds and power modes.
- Try to focus more on accuracy rather than on output tokens.
- Try this pipeline on various other lightweight models like Qwen, Llama and Gemma.

References

[1] E. J. Husom, A. Goknil, M. Astekin, L. K. Shar, A. Kasen, S. Sen, B. A. Mithassel, and A. Soylu, "Sustainable llm inference for edge ai: Evaluating quantized llms for energy efficiency, output accuracy, and inference latency," *arXiv preprint arXiv:2504.03360*, 2025.

[2] M. Arya and Y. Simmhan, "Understanding the performance and power of llm inferencing on edge accelerators," *arXiv preprint arXiv:2506.09554*, 2025.

[3] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "Llm.int8(): 8-bit matrix multiplication for transformers at scale," *arXiv preprint arXiv:2208.07339*, 2022.

[4] E. J. Husom, A. Goknil, L. K. Shar, and S. Sen, "The price of prompting: Profiling energy use in large language models inference," *arXiv preprint arXiv:2407.16893*, 2024.

[5] F. Caravaca, A. Cuevas, and R. Cuevas, "From prompts to power: Measuring the energy footprint of llm inference," *arXiv preprint arXiv:2511.05597*, 2025.

[6] C. Team, "Track carbon emissions from ml computing," 2023, [Online]. Available: https://mlco2.github.io/codecarbon/.

[7] U. E. Dammu, "Energy efficiency in llm inference: Open vs. closed models," 2025, mSDS 692 Practicum I, Regis University. [Online]. Available: https://github.com/Mrsnellek/LLM_Research_Group/tree/main/Emmaniuel.

[8] NVIDIA Corporation, "Jetson nano developer kit technical reference manual," 2023, [Online]. Available: https://developer.nvidia.com/embedded/jetson-nano.

[9] ——, "tegrastats utility documentation," 2023, [Online]. Available: https://developer.nvidia.com/embedded/jetson.

[10] T. Team, "Tinyllama-1.1b-chat-v1.0," 2024, [Online]. Available: https://huggingface.co/TinyLlama/TinyLlama-1.1B-Chat-v1.0.

[11] Ollama, "Ollama model library," 2025, [Online]. Available: https://ollama.com/library.

[12] ServiceNow, "Repliqa dataset," 2025, [Online]. Available: https://huggingface.co/datasets/ServiceNow/repliqa.

[13] corag, "Multihopqa (hotpotqa configuration)," 2025, [Online]. Available: https://huggingface.co/datasets/corag/multihopqa.

[14] C. Team, "Ccsum summarization dataset," 2024, [Online]. Available: https://huggingface.co/datasets/ccsum/ccsum_summary_only.

[15] P. Technologies, "Interactive data visualization in python," 2024, [Online]. Available: https://plotly.com/python.

[16] S. Lin, J. Hilton, and O. Evans, "Truthfulqa: Measuring how models mimic human falsehoods," 2022, [Online]. Available: https://github.com/sylinrl/TruthfulQA.

[17] OpenAI, "Gpt-5 pro and gpt-4.1 mini large language models," 2025, [Online]. Available: https://platform.openai.com.

[18] Anthropic, "Claude: Ai assistant by anthropic," 2025, [Online]. Available: https://www.anthropic.com/claude.

[19] F. P. et al., "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[20] T. W. et al., "Transformers: State-of-the-art natural language processing," in *Proceedings of EMNLP: System Demonstrations*, 2020, pp. 38–45.

[21] W. McKinney, "Data structures for statistical computing in python," in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61.

[22] J. D. Hunter, "Matplotlib: A 2d graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.